

Rapport de projet

Soutenance finale



Clément Bonaud, Mathilde Pujol-Sillié, Mathilde Hubert, Anthon Nazon, Théo Privat

Plan:

1. Le contexte du projet

- L'origine du projet
- La nature du projet

2. Les objectifs

- Intérêts du projet
- Objectifs techniques
- Objectifs personnels

3. Le projet

- Le type du projet
- Les mécaniques
- L'histoire

4. Qui sommes-nous ?

- Notre entreprise
- Notre équipe
- Notre façon de travailler

5. L'organisation

- Nos méthodes d'organisation
- Le planning
- Les étapes importantes

6. Architecture technique

- Architecture générale
- Le rôle du serveur
- Le rôle du client
- Communication
- Sérialisation
- Synchronisation

7. Histoire du monde

8. Processus création graphique

9. Choix technique

10. Présentation technique chronologique

- Description des tâches
- Problèmes rencontrés

11. État lors de la dernière soutenance

12. Projection future

13. Nouvelles connaissances acquises

14. Les annexes:

- Les ressources graphiques
- Les captures d'écran

Détails de la partie 6:

1. Création de l'environnement python avec pygame pour afficher des formes géométriques
2. Mise en place du projet à l'aide de classes (POO)
3. Création de la partie serveur et client séparé (GameState)
4. Multijoueur avec connection sockets
5. Mise en place du github avec readme
6. Gamemanager pour une logique centralisée
7. Base des pnj et ennemis
8. Caméra
9. EntityManager
10. Sort lançable par le joueur via bouton de la souris (gestion des événements)
11. Murs
12. Hitbox
13. Collisions
14. Système d'ui dynamique
15. Tentative interpolation
16. Événement de collisions
17. IA
18. Système de vie/dégât
19. Collision ennemi/PNJ avec mur
20. IA manager
21. Déplacement via pathfinding
22. Base/prototype pour reconnaissance vocale
23. Animateur/sprite
24. Optimisation du pathfinding(en cours)
25. Pathfinding bugs (en cours de résolution)
26. Carte
27. Logique de tilemap
28. Website
29. Ressources graphiques
30. L'histoire
31. Différentes couches du donjon
32. Porte avec interaction pour changer de couches
33. Création aléatoires des murs dans les couches
34. Création aléatoire des ennemis dans les couches
35. Ajout d'un filtre pour diminuer le rayon de vision
36. Adaptation ias aux système couches
37. Ajout des types de sorts
38. Barre de vie
39. Résolution bug reconnaissance vocale
40. Résolution bug network

41. Ajout d'un système de debug pour le pathfinding
42. Résolution bug pathfinding
43. Ajout des types d'ennemis
44. Optimisation path finding
45. Modifications comportement des ias
46. Ajout d'animations pour les sorts
47. Ajout d'animations pour les ennemis
48. Graphismes ennemis et PNJ
49. Graphismes sorts
50. Modifications site
51. Événement de mort
52. Affichage dialogue des PNJ
53. Énigmes des PNJ
54. Système de sauvegarde
55. Création exécutable
56. Plein écran
57. Menu paramètre
58. Cycle jour/nuits
59. Affichage donjon (barre de vie boss + info niveau actuel)
60. Système de son/musique
61. Optimisation serveur avec envoie par couche
62. Ecran de chargement
63. Page de crédit
64. Correction problème serveur
65. Correction de synchronisation
66. Player respawn
67. Ajout de sorts
68. amélioration des menus
69. settings
70. Système de quêtes
71. Équilibre
72. Graphisme
73. Site web
74. Boss
75. amélioration ias ennemis
76. effets de ralentissement

Contexte du projet

- Origine et nature du projet

Au début de notre projet, nous avons hésité sur le type de jeu que nous souhaitions concevoir, notamment entre rôle playing game et incremental game. Incremental game est un type de jeu dans lequel le joueur doit faire progresser une ressource, généralement de l'argent, pour ensuite acheter des améliorations qui lui permettront de gagner encore plus de cette ressource. Cela est généralement automatique, il y a même des revenus passifs.

Cependant lorsque nous nous sommes mis d'accord lors de la première réunion pour savoir vers quel jeu nous voulions nous tourner, le groupe a décidé de faire un RPG car cela pourrait apporter plus de possibilités qu'un incremental game.

Nous nous sommes interrogés sur un autre point : est-ce que nous allions concevoir un jeu en 2D ou en 3D. Si nous partions sur de la 2D nous utiliserions pygame alors que pour la 3D, deux choix s'offraient à nous, Ursina ou Panda3D.

En vérifiant les ressources de ces deux moteurs de jeux, au sein du groupe nous avons trouvé Panda3D un peu plus complexe à utiliser. Nous n'avions donc plus que deux choix possibles : la 3D ou la 2D. Nous sommes partis vers de la 2D, car cela nous a semblé plus accessible à tous.

Pour le sujet du projet, dès le début nous savions que nous voulions nous orienter vers le monde de la magie mais c'était encore trop vague. C'est effectivement un domaine très vaste qui demande quelques spécificités pour se centrer vers un point et ne pas s'éloigner de ce qui est initialement prévu.

Une fois le type de jeu et l'univers décidés, nous avons commencé à noter quelques idées afin de nous mettre d'accord sur une seule et même histoire et de pouvoir développer ces idées plus tard si besoin.

Suite à cela, il a fallu trouver un nom qui corresponde à l'esprit du jeu mais aussi qui plaise au groupe. Le jeu s'est donc appelé "An bra da cadavra" mais ce nom ne plaisait pas forcément au groupe entier. Cependant, durant une courte période, ce fut le nom du jeu car nous n'avions pas d'inspiration. Mais lors d'une des réunions, une question est revenue sur le nom initialement donné, ce qui a mené à de nouvelles propositions. L'une d'elles a fait l'unanimité parmi le groupe, "Avada-Kedavoix" qui est désormais le nom officiel de notre projet.

Objectifs du projet

- Intérêts du projet

Ce projet nous permettra d'atteindre de multiples objectifs, tous importants pour notre carrière professionnelle. Il va nous aider à nous améliorer professionnellement et techniquement. Concrètement, le but final de ce projet est de créer un jeu pour ordinateur intégrant un système d'intelligence artificielle et un système réseau. Nous réalisons ce projet pour apprendre de nouvelles compétences techniques, par le développement. De plus, par la communication et l'organisation en équipe dans un objectif commun, ce projet va nous apprendre des compétences sociales.

- Objectifs techniques

Pour ce projet nous avons comme objectifs techniques, la mise en place d'un système d'intelligence artificielle, d'un système de réseau, d'un système de physique avec des collisions, d'un système de reconnaissance vocale et d'autres ajouts mineurs. Dans un premier temps, nous nous consacrerons à la mise en place du système de multijoueur via le réseau. Ce système constitue le squelette de l'intégralité du jeu car toute action du joueur et du monde autour de celui-ci devra être adaptée pour ce système. Cette partie constitue le squelette car elle doit permettre à tous les joueurs d'être synchronisés. L'objectif d'un tel module est d'apprendre à organiser une architecture plus complexe qu'un simple jeu sans cette partie multijoueur. Nous rajouterons de la physique au monde, c'est-à-dire que pour empêcher le joueur et autres objets de traverser des murs ce module doit être créé. Il doit aussi détecter les collisions entre des objets et le joueur. Les créatures devront être animées d'une intelligence artificielle pour gérer leur déplacement et leur comportement vis-à-vis du joueur comme les ennemis ou les villageois. Pour prolonger l'expérience utilisateur, l'objectif est de développer un système d'équipement et de progression pour augmenter la difficulté au cours du temps. Et pour finir, pour nous démarquer des autres jeux de ce genre, nous devons développer un système de reconnaissance vocale. Ce système marche grâce à des outils fonctionnant via des intelligences artificielles.

- Objectifs personnels

Tous ces objectifs ont pour but de nous améliorer dans l'exploitation d'un langage de programmation. Par exemple, grâce aux recherches que nous devons faire pour trouver des explications et définitions dans des documentations. Ceci va nous apprendre à savoir où chercher des informations quand on est perdu et à améliorer notre compréhension du code.

Nous apprendrons aussi à gérer un projet de son architecture technique à son architecture dans le code. Ceci implique aussi l'apprentissage des conventions de programmation pour assurer une cohérence dans les lignes écrites entre tous les membres du groupe. Au-delà des aspects techniques, ce projet vise également le développement de compétences sociales, notamment la capacité à travailler en équipe, à communiquer efficacement et à s'organiser autour d'un projet commun comportant des tâches variées.

Le projet

Avada-Kedavoix est un RPG où chaque personnage incarne un magicien en 2D. Les joueurs doivent unir leurs pouvoirs et leurs voix pour repousser les ennemis. Vous incarnez un Écho-sorcier et apprenez à canaliser la magie à travers votre voix, plus votre maîtrise est grande, plus vos sorts deviennent puissants. Affrontez les ténèbres dans la Tour des Abysses, une structure inversée plongeant directement dans les Enfers. Coopérez avec d'autres joueurs pour affronter des créatures infernales : démons, zombies, squelettes et autres abominations qui hantent les profondeurs de la tour. Avancez dans le labyrinthe, prenez garde à ne pas vous égarer. Vous devrez pour progresser, discuter et marchander des armes et des denrées dans le village avec les différents marchands et personnages non-joueurs. Il n'y a aucun affrontement entre joueurs, seule la magie collective permet de triompher du mal.

Notre jeu est un RPG, signifiant role play game. Dans ce type de jeu, le joueur incarne un personnage fictif, et très souvent son but est d'avancer dans son aventure en éliminant des ennemis de plus en plus fort, le tout en s'améliorant pour progresser toujours plus loin.

Dans notre jeu, le but est d'affronter des ennemis pour progresser. Mais sa particularité, c'est la présence de la Tour des Abysses. Il s'agit d'un donjon composé de plusieurs étages. Dans chaque étage se trouvent des monstres mais aussi des pièges. Pour progresser dans cette tour, le joueur va devoir utiliser sa voix pour lancer des sorts.

Qui sommes-nous ?

- Notre entreprise

Nous sommes Wizz Studio, une entreprise de création de jeux vidéo. Nous sommes actuellement en train de développer un nouveau jeu vidéo nommé Avada-Kedavoix.

Wizz à été choisi pour le mot wizard (en anglais sorcier) qui est un thème récurrent pour tous nos jeux actuels et futurs, en cours d'élaboration ou simplement à l'état de simples idées pour le moment. Ce nom est en anglais afin de pouvoir toucher un public bien plus large que celui de la France. Dans le jeu Avada-Kedavoix, le ou les joueurs incarnent ces sorciers.

- Notre équipe

Nous sommes une équipe de 5 personnes dont:

- Clément Bonnaud, le chef de projet : il est donc responsable de l'organisation et de surveiller l'avancement du projet. Il guide et rassemble les membres, organise les réunions et s'assure que nous rendons bien tout le travail lorsqu'il le faut. Il travaille beaucoup sur la partie intelligence artificielle et les recherches algorithmiques notamment A*. Il s'occupe aussi de la partie physique et traitement vocal grâce à son expérience en développement de jeu Python.
- Mathilde Pujol-Sillié, la responsable des rendus écrits et des comptes rendus de réunions. Elle s'assure de garder une trace claire et normalement sans fautes d'orthographe de tout ce qui a été fait ou prévu. Elle est également responsable de tout le côté rédaction, donc majoritairement des comptes rendus des cahier des charges, ainsi que de la relecture de tout ce qui est diaporama par exemple, pour essayer au maximum d'évincer les erreurs d'orthographe, ce qui correspond bien à son profil puisqu'elle aime écrire. Elle est en charge de toutes les parties dialogues et histoires du jeu et de toutes les interfaces liées à l'histoire. Elle s'occupe également d'une partie du système d'intelligence artificielle, du site web et de la coloration de certains graphismes.
- Mathilde Hubert, la responsable du temps ainsi que la graphiste du projet. Elle veille à ce que l'avancement du projet respecte les délais

impartis, c'est grâce à elle que nous arrivons à avoir une idée de où est-ce que nous en sommes, des objectifs qu'il faudrait atteindre, sur quels points nous sommes en avance ou en retard. Elle s'occupe également de concevoir certains graphismes, tels que les sorts, en s'appuyant sur ses compétences en dessin. Elle a déjà utilisé le langage Python lors de la réalisation de projets en équipe au lycée.

- Anthon Nazon, la personne qui s'occupe de la partie communication du jeu. Il s'occupe de la présentation du jeu, et s'assure de communiquer les informations à l'intérieur du groupe et de parler de nos projets et jeux. Il explique notre jeu pour donner envie aux gens de jouer. Il s'occupe également de la physique du jeu et des cartes et participe à la conception des cartes du jeu en s'associant avec la graphiste du groupe. De plus, il cherche certaines bibliothèques de modèles graphiques déjà existants afin de nous faciliter quelques tâches.
- Théo Privat, le développeur en chef du projet. C'est le membre du groupe avec la plus grande expérience, il a configuré le GitHub du projet et toutes les explications pour aider les autres membres. Il conçoit l'architecture centrale du jeu afin de fournir une base solide pour l'ensemble des fonctionnalités. Il maîtrise le langage Python et les concepts clés du développement de jeux vidéo, tels que le réseau, la physique et l'intelligence artificielle. Il possède également une expérience dans la création de jeux vidéo acquise lors du développement d'un jeu mobile avec Unity.

- Notre façon de travailler

Nous travaillons préférentiellement de manière individuelle, en se répartissant les tâches, en fonction de nos compétences mais aussi des aspects et techniques que nous avons le plus envie d'apprendre. Certaines tâches sont réalisées en binôme, voire parfois en trinôme par soucis de temps principalement mais aussi pour varier les idées et les savoirs afin d'obtenir le jeu qui correspond au maximum à la vision de chaque personne. Beaucoup de méthodes dont nous avons besoins n'étant pas enseignées en cours, nous nous appuyons sur des ressources extérieures, principalement des leçons et des vidéos en ligne mais aussi en demandant à d'autres personnes plus expérimentées pour trouver ce dont nous avons besoin. Afin que personne ne soit livré à son sort, nous nous assurons d'organiser régulièrement des réunions pour que tout le monde sache où est-ce que nous en sommes et quels

sont les points qui posent un problème. Nous utilisons GitHub afin de mettre tout notre travail en commun.

Organisation

- Méthodes d'organisation

Afin de bien nous organiser pour la réalisation de ce projet de groupe, nous avons choisi d'utiliser l'application Miro, qui nous permet de suivre l'avancement des tâches en temps réel, de noter nos prévisions pour les prochaines soutenances, de visualiser la carte mentale de notre jeu, ainsi que de définir nos objectifs pour cette première soutenance.

De plus, pour chaque document devant être rédigé, tels que le cahier des charges fonctionnelles ou ce rapport de soutenance, nous avons créé des documents partagés. Cela permet à chacun de rédiger sa partie tout en tenant compte du travail des autres, mais aussi de s'entraider si nécessaire sur certaines parties.

Pour faciliter la communication au sein de l'équipe, nous avons décidé de créer un groupe sur Instagram, ainsi qu'un autre sur Discord. Cela nous permet d'échanger rapidement et de nous entraider en cas de problèmes.

Enfin, nous avons organisé des réunions régulièrement afin de suivre l'avancement de la conception du projet et de discuter ensemble des points importants. À la suite de chaque réunion, nous avons rédigé un compte rendu afin de garder une trace des points abordés, ainsi que des modifications décidées, que ce soit en termes d'organisation ou concernant le fonctionnement du jeu.

- Planning

Au début de l'année, nous avons créé un planning sur lequel nous avons noté toutes les dates importantes pour ce projet, ainsi que les dates de nos réunions. Vous pourrez trouver ce planning dans les annexes.

- Étapes importantes

Au début du projet, il était essentiel de bien organiser le travail au sein du groupe, notamment en ce qui concerne la répartition des tâches. Nous avons donc organisé une réunion durant laquelle nous avons fait une liste de toutes les tâches à réaliser pour le projet.

Nous avons ensuite essayé d'évaluer la difficulté et la charge de travail que représentait chacune de ces tâches. La répartition s'est faite collectivement, en tenant compte des compétences de chacun, mais aussi de l'envie d'apprendre et de découvrir de nouveaux aspects du projet. Par exemple, Mathilde Pujol étant particulièrement à l'aise avec l'écriture, nous avons décidé de lui confier la rédaction du storyboard.

Chaque membre du groupe est ainsi devenu responsable d'une tâche principale, tout en étant suppléant sur au moins une autre. Cela nous permet de nous entraider en cas de difficulté ou d'absence.

Enfin, après la répartition des tâches, nous avons mis en place un suivi régulier grâce à Miro, afin de vérifier l'avancement du projet et d'ajuster l'organisation si nécessaire. Nous pouvons ainsi voir si une tâche est terminée, en cours de réalisation ou non commencée et finalement, nous pouvons nous assurer que le projet avance dans les délais impartis.

Architecture technique:

Architecture générale:

Le projet repose sur une approche client-serveur. Le serveur détient la version de référence du monde, et chaque client affiche une copie locale mise à jour en continu. Le client ne “dessine” pas directement le monde complet sans données serveur : il reçoit des snapshots et applique les changements localement.

Le rôle du serveur:

Le serveur une fois lancé écoute les connexions entrantes et va soit initialiser un nouveau monde soit le charger depuis une sauvegarde sous format json. Une fois lancé, le serveur a 2 fonctions principales.

- Il va dans un premier temps recevoir et envoyer des messages avec les clients, par exemple quand un joueur se connecte, le serveur va envoyer au joueur une copie du monde mais seulement sur sa propre couche (la couche correspond à 1 à la surface et $N+1$ au niveau N). Par la suite, le joueur va recevoir les éléments qui ont changé par rapport à la dernière version reçue. Ces mises à jour sont envoyées 30 fois par seconde pour garder une synchronisation entre les joueurs.
- Ensuite le serveur a aussi pour rôle de gérer le monde c'est lui qui va s'occuper de mettre à jour les entités avec leur collision. Il va s'occuper entièrement des sorts, des PNJ, des ennemis et des murs et le donjon.

Le rôle du client:

Le client s'occupe localement du rendu du jeu et de la communication avec le serveur. Toute la logique d'affichage passe par le GameManager, qui initialise Pygame et centralise la boucle de jeu. C'est également lui qui gère les composants de l'interface, la caméra, ainsi que les entrées du clavier, de la souris et de la voix.

Le client envoie au serveur les informations liées au joueur local, notamment sa position et ses actions. En retour, il reçoit une copie du monde qu'il met à jour localement, puis qu'il affiche avec Pygame. Cette séparation entre logique serveur et affichage client permet d'avoir un jeu plus stable, plus lisible et plus facile à synchroniser en multijoueur.

Communication:

La communication entre le client et le serveur repose sur un système de messages typés. Chaque requête contient un type et des données associées, ce qui permet de traiter facilement les différents événements du jeu. Le protocole est construit de

manière à être simple à lire, à transmettre et à étendre si de nouvelles fonctionnalités sont ajoutées par la suite.

Pour éviter les erreurs de transmission, chaque message est encapsulé avec une taille dans un en-tête, puis avec un payload sérialisé. Cette méthode permet au receveur de savoir exactement combien d'octets lire avant de reconstruire l'objet reçu. Les entités elles-mêmes ne sont pas envoyées directement sous forme d'objets Python complets, elles sont transformées en dictionnaires grâce à la classe de sérialisation, ce qui évite d'envoyer des attributs inutiles ou incompatibles avec le réseau.

Les principaux échanges utilisés dans le projet sont les suivants :

- CONNECT pour attribuer l'identifiant du joueur à sa connexion
- GAME_STATE pour transmettre l'état du monde
- PLAYER_UPDATE pour envoyer la position du joueur
- PLAYER_CAST_SPELL pour créer un sort
- PLAYER_UPDATE_SPELL pour synchroniser certains états d'un sort
- CHANGE_LAYER pour gérer le passage d'un étage à un autre
- PLAYER_RESPAWN pour replacer un joueur à sa réapparition
- DUNGEON_DATA pour transmettre les données de génération du donjon

Sérialisation

Pour faire communiquer le client et le serveur, il a d'abord fallu transformer les objets du jeu en données transmissibles par le réseau. En effet, il est impossible d'envoyer directement des objets Python complexes contenant des éléments graphiques ou des références internes à Pygame. C'est pour cela que nous avons mis en place une classe de sérialisation commune, utilisée par les différentes entités du jeu. Cette classe permet de convertir un objet en dictionnaire, en ne conservant que les attributs utiles à la logique du jeu, et en supprimant les éléments non transmissibles comme les surfaces, les rectangles ou d'autres objets liés à l'affichage. Le but est de garder uniquement les informations nécessaires pour reconstruire l'objet de l'autre côté du réseau. Cette logique est définie dans la classe Serializable et tous les éléments qui doivent transiter entre le client et le serveur héritent de cette classe comme les classes Player, Enemy et PNJ.

La sérialisation repose sur deux opérations complémentaires. D'un côté, la méthode `to_dict` permet de transformer une entité en dictionnaire exploitable. De l'autre, la méthode `from_dict` permet de recréer une instance à partir de ces données. Ce fonctionnement rend l'échange d'informations plus simple, plus générique et plus facile à étendre si de nouvelles classes sont ajoutées au projet. Par exemple, si nous créons un nouveau type de sort, il suffit qu'il respecte le même principe de sérialisation pour qu'il puisse être transmis au serveur et aux clients sans refonte

complète du système. Nous avons aussi ajouté un système de différence d'état avec `diff_to_dict`, afin de n'envoyer que les attributs modifiés quand cela est possible, ce qui limite la quantité de données transmises. La classe de base gère donc à la fois la conversion complète et la mise à jour partielle.

Synchronisation:

La synchronisation consiste à faire en sorte que tous les clients partagent une vision cohérente du monde, tout en conservant la logique centrale côté serveur. Le serveur est l'autorité principale, il met à jour les entités, calcule les collisions, fait avancer le temps de jeu, puis diffuse régulièrement l'état du monde vers les clients. Cette logique est regroupée dans `gameState.py`, où l'on trouve les différents gestionnaires d'entités et les fonctions de mise à jour globales. Le serveur envoie ensuite les données sous forme de messages typés.

Côté client, la synchronisation repose sur la réception continue des états envoyés par le serveur. Le `ClientManager` écoute les messages réseau, puis applique les mises à jour reçues à la copie locale du monde. C'est ce mécanisme qui permet d'afficher les autres joueurs, les ennemis, les sorts, les PNJ et les murs avec un état proche de celui du serveur. Lorsqu'un joueur se connecte, il reçoit d'abord un état complet du monde, puis seulement les mises à jour suivantes. Lors d'un changement d'étage, d'une réapparition ou d'une génération de donjon, le serveur renvoie un état complet adapté à la couche concernée.

La synchronisation n'est pas seulement globale, elle est aussi adaptée à la couche du joueur. Le serveur ne diffuse pas toujours le même monde à tout le monde, il filtre les données selon la couche active pour n'envoyer que les éléments pertinents. Cela évite d'afficher ou de traiter des objets inutiles et permet de gérer le donjon par niveaux. De plus, pour conserver une sensation de fluidité, le joueur local garde certains calculs en local, notamment son déplacement et le centrage de la caméra, tandis que le serveur reste responsable de la validation de l'état général. Cette séparation entre calcul local et autorité serveur permet de conserver un jeu réactif sans perdre la cohérence du multijoueur.

Histoire du monde

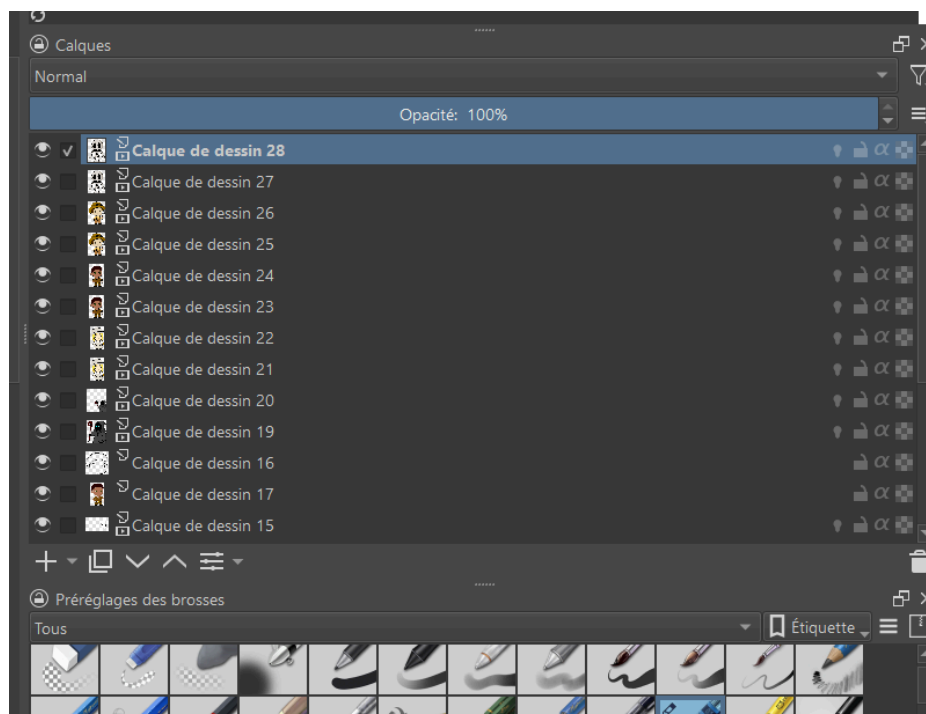
Le monde dans lequel les joueurs évoluent dans le RPG, est un univers fantastique avec un décor qui rappelle plutôt l'époque médiévale. On y retrouve outre les sorciers qui sont les personnages principaux, plusieurs espèces mystiques dont par exemple des gnomes et des dragons, comme mentionnées dans les parties dédiées. L'idée principale de cet univers est de plonger le joueur dans un monde différent, rempli de mystères et de magie. L'utilisation des commandes vocales est également conçue pour apporter plus de réalisme à l'expérience de jeu.

Processus création graphique

Pour réaliser un graphisme, nous avons d'abord cherché des images sur internet pour trouver de l'inspiration. Puis, nous avons dessiné un premier brouillon sur un papier afin d'avoir une première base sur laquelle travailler. Une fois le brouillon terminé, nous avons redessiné le brouillon sur un ordinateur grâce à une tablette graphique.



Nous avons installé le logiciel krita pour utiliser la tablette graphique. Cela nous a permis d'utiliser différentes brosses pour dessiner, colorier et flouter les dessins. Nous avons travaillé avec différents calques de dessins afin de superposer des couches du dessin sans modifier les parties déjà terminées tels que les tracés principaux du dessin. Par exemple, il y a un calque dédié au tracé du dessin en noir et blanc alors qu'un autre sert simplement à la couleur.



Après avoir obtenu le dessin final, nous avons ajouté la couleur en commençant par les couleurs principales pour finir avec les couleurs plus sombres des ombres.

Nous avons répété ce processus pour chacune des ressources graphiques. La dernière étape fut d'ajouter les animations à chaque dessin. Pour cela, nous avons repris l'image principale que nous avons dessinée et nous l'avons copiée plusieurs fois. Puis, nous avons sélectionné certaines parties du corps et à l'aide de l'outil transformer nous les avons déplacées et tournées pour donner un effet de mouvement.

Finalement, nous avons obtenu le résultat suivant:



Nous avons également utilisé des ressources graphiques trouvées sur Craftpix pour le sorcier représentant le joueur.

Choix techniques

Moteur de jeu:

- Pygame a été utilisé comme bibliothèque principale pour le développement du jeu, car elle fournit un ensemble d'outils adaptés à la création d'un jeu en 2D. Elle permet de gérer facilement l'affichage, les événements clavier et souris, les animations, les collisions ainsi que le rendu des éléments graphiques. Son intégration avec Python est simple et elle offre suffisamment de possibilités pour développer un projet complet tout en restant accessible.

Outils de collaborations:

- GitHub nous a permis de centraliser le code et de permettre à tous les membres du groupe de travailler indépendamment sur leur fonctionnalité dans des branches spécifiques pour chaque élément du jeu. Par la suite ils nous suffit de merge les branches de développement dans la branche principale
- Miro nous a permis de planifier ce que nous devons faire et répartir la charge de travail aux différents membres du groupe.

IA:

- L'algorithme A* a été retenu pour le pathfinding des ennemis car il permet de calculer un chemin efficace vers une cible tout en limitant les calculs inutiles. Son fonctionnement repose sur une recherche guidée par la distance restante à parcourir, ce qui le rend plus pertinent qu'une recherche brute dans un environnement avec des collisions et des obstacles. Ce choix est adapté à un jeu en temps réel, car il offre un bon compromis entre précision, rapidité et simplicité d'implémentation.
- Vosk a été retenu comme choix technique pour la reconnaissance vocale en raison de sa légèreté, de son fonctionnement hors ligne et de sa facilité d'intégration dans un projet Python. Cette solution open source propose des modèles compacts, compatibles avec de nombreuses langues dont le français, ce qui la rend adaptée à une application nécessitant une reconnaissance vocale fiable sans dépendance à un service en ligne. Son utilisation permet également de conserver de bonnes performances sur des machines modestes, tout en simplifiant l'implémentation grâce à son module Python et la possibilité de l'utiliser via un flux audio. Ce choix répond donc à un objectif de simplicité, de portabilité et d'efficacité.

Exécutable:

- Pyinstaller: Nous avons choisi PyInstaller pour créer un exécutable. Son utilisation a été assez simple après la recherche de quelques documentations et exemples en ligne pour créer le fichier .spec. PyInstaller permet de regrouper le code Python, les dépendances et les ressources nécessaires dans un exécutable. Il a néanmoins fallu adapter la configuration pour gérer correctement les chemins d'accès, les modules du projet et certaines librairies externes. L'exécutable obtenu permet de lancer le jeu directement, sans avoir à exécuter manuellement le script Python principal.

Cartes Tiled :

- PyTMX est utilisé pour charger et lire les cartes créées avec Tiled. Ce choix permet d'exploiter facilement les fichiers au format TMX dans un projet Python, tout en conservant la structure de la carte, les calques et les objets définis dans l'éditeur. Il s'agit d'une solution pratique car elle facilite l'intégration des cartes dans le moteur du jeu et permet de séparer la création des niveaux de leur utilisation dans le code.

Flux audio:

- Sounddevice a été choisi pour la gestion de l'entrée audio liée à la reconnaissance vocale. Cette bibliothèque permet de récupérer simplement le flux audio provenant du micro afin de l'envoyer au système de reconnaissance vocale. Elle est légère, facile à intégrer et fonctionne bien avec les autres outils utilisés dans le projet, notamment Vosk. Son utilisation facilite la capture du son en temps réel sans ajouter une complexité excessive au code, ce qui en fait une solution adaptée à notre besoin.

Connexion serveur/client:

- La connexion sockets a été utilisée pour mettre en place la communication entre le client et le serveur. Ce choix permet d'échanger des informations en temps réel sur l'état du jeu, comme la position des entités, les collisions, les dégâts ou encore certaines actions du joueur. L'utilisation des sockets offre une communication directe et réactive, ce qui est essentiel pour un jeu multijoueur ou reposant sur une architecture client/serveur. Ce système constitue donc la base du fonctionnement réseau du projet.

Traitement des données:

- JSON a été utilisé pour stocker et échanger certaines données du projet, notamment celles liées à la sauvegarde. Ce format est pratique car il est simple à lire, à écrire et à manipuler en Python. Il permet de conserver des informations structurées tout en restant léger et facilement exploitable par le jeu. Son utilisation facilite donc la gestion des données persistantes et rend leur traitement plus clair dans le code.

Graphismes:

- Krita est le logiciel que nous avons utilisé pour réaliser nous-mêmes les graphismes. Ce logiciel permet d'avoir accès à de nombreuses brosse différentes gratuitement et d'utiliser des calques de dessin.

Présentation technique dans l'ordre chronologique:

1. Environnement Python avec Pygame: Pour avoir une idée de ce que nous allions faire, nous avons commencé par créer une scène avec pygame vierge. Cette scène a été remplie de formes géométriques de différentes couleurs pour commencer à avoir une idée de comment fonctionne Pygame. Ceci nous a permis de nous projeter pour savoir quelle architecture prendre pour le développement. Nous avons créé un personnage simple, représenté par une sphère qui se déplace grâce aux touches du clavier. Cet environnement était simple et minimaliste mais nous l'avons fait évoluer.
2. Nous avons choisi d'utiliser l'approche en POO ou programmation orientée objet. Ce paradigme est très utile dans la création de jeux car elle permet d'isoler des fonctionnalités dans des classes où chacune a un rôle bien précis. Cela permet de créer plusieurs objets/entités avec la même logique simplement. Par exemple, tous les joueurs possèdent la même classe joueur, simplifiant l'architecture du code. Nous avons créé des "Manager" qui s'occupent de gérer une partie précise de la logique, par exemple le networkManager qui s'occupe du réseau. Ce paradigme bien qu'utile a posé un problème, tous les membres de l'équipe ne connaissaient pas cette manière de coder et ils ont donc dû apprendre pour pouvoir comprendre ce qui avait été fait et par la suite comment participer au projet.
3. Nous avons par la suite commencé le multijoueur. En s'aidant de contenus sur YouTube, nous avons créé un serveur sockets qui s'occupe de gérer les connexions. Nous avons aussi rajouté la partie pour que les clients rejoignent le serveur. Mais concrètement, rien ne s'est passé, il a donc fallu que le serveur et le client communiquent. Pour se mettre d'accord, le serveur et le client se partagent une copie du monde, le "GameState". Dans cette copie se trouve l'ensemble des éléments du jeu puisque tous les joueurs, tous les PNJ et tous les ennemis sont listés. Pour la protection de ces données, seul le serveur a le droit de les modifier. C'est donc trente fois par seconde que le serveur envoie à tous ses clients l'état du monde. Tous les clients vont par la suite dessiner tous ces éléments à leur place, permettant à tous les joueurs de voir un seul et même monde. Cette partie a posé beaucoup de problèmes, notamment sur le déplacement des joueurs. Comme seul le serveur a le droit de modifier l'état du monde, les joueurs ne peuvent se déplacer que localement. Pour permettre au joueur de se déplacer aussi du côté du serveur, il a fallu que le joueur envoie sa nouvelle position au serveur pour qu'il l'enregistre. Cela permet à tous les autres joueurs de voir où nous sommes.

4. Pour gérer les requêtes, des ajustements ont dû être faits. Par exemple, pour simplifier l'envoi et la réception de requêtes, elles sont encapsulées grâce à la classe Message qui leur attribue un type, son payload (l'information qui doit être envoyée) et en header la longueur de la requête. Le header permet d'éviter des erreurs en délimitant l'information et éviter de dépasser sur celle d'après ou même éviter d'oublier un morceau de l'information. Cette longueur est inscrite dans les premiers octets du message. De plus, ces requêtes possèdent un type, ce qui va permettre de traiter plus simplement l'information. Par exemple il existe les types: CONNECT, DISCONNECT, PLAYER_UPDATE, PLAYER_CAST_SPELL. Et pour finir, il y a le payload ou la partie utilisable. Il s'agit de l'information qui est envoyée, mais pour pouvoir envoyer de l'information via socket, il est compliqué et peu recommandé d'envoyer les objets Python directement via le réseau. Pour l'envoi du gamestate par exemple, nous n'allons pas envoyer tous les objets Python en même temps. C'est pour cela que nous sérialisons les instances des classes. Pour ce faire, chaque classe qui doit transiter entre le client et le serveur se voit hériter de la classe Serializable qui permet d'automatiquement sérialiser les attributs et supprime les méthodes pour ne garder que les valeurs importantes. Le serveur avant d'envoyer le GameState va donc sérialiser, transformer en texte, toutes les instances de classe via la méthode to_dict héritée de la classe Serializable. Puis le client va faire l'inverse de son côté il va récupérer la requête qu'il va désérialiser via la méthode from_dict pour récupérer l'objet en Python à partir du texte et pouvoir le traiter plus simplement. Cette méthode de transformer toutes les instances de classe en json permet de simplifier grandement le développement car elle est universelle. Par exemple, si nous voulons créer une nouvelle classe comme Enemy2 nous n'avons pas besoin de tout refaire mais simplement de la faire hériter de la classe Serializable.
5. Une fois la base de Pygame et du multijoueur faite, nous avons créé le dépôt GitHub et les fichiers [README.md](#), [CONTRIBUTING.md](#) et [CODE_IMPROVEMENTS.md](#). Ces fichiers générés grâce à l'IA permettent d'aider ceux qui ont du mal à mettre en place un projet en les guidant étape par étape pour l'installer, installer les dépendances et lancer le projet. Ces fichiers permettent aussi d'expliquer comment concrètement participer au projet, en enseignant comment contribuer, comment créer une branche pour une fonctionnalité et comment respecter certaines conventions comme celle de nommage des variable et le fait de bien documenter le code que nous écrivons. Toutes ces aides permettent de maintenir le projet le plus propre possible, ainsi que permettre à tout le monde de comprendre le code des autres sans devoir perdre du temps à le comprendre. Et enfin ces fiches permettent de suivre l'état du projet en listant les fonctionnalités déjà présentes et celles qu'il reste à développer.

6. Pour centraliser la logique du côté du client, nous avons créé le GameManager qui centralise tous les éléments du côté client. Il s'agit de la classe la plus complète. Il s'occupe de mettre en commun les requêtes du serveur, de dessiner les éléments, de redistribuer les événements Pygame aux modules qui en ont besoin et de mettre à jour le monde local.
7. Nous avons par la suite créé une base pour les ennemis et pour les PNJ vierge pour tester si l'ajout de nouvelles entités complexifie le code. Mais après des ajustements, l'ajout de nouvelles classes se fait assez simplement.
8. Le joueur pouvait se déplacer et les autres clients le voyaient se déplacer mais la caméra était fixe et le joueur pouvait sortir du champ de vision de la caméra. C'est pour cela que nous avons dû mettre en place la caméra. Mais les caméras en Pygame n'existent pas réellement. Nous avons donc dû rajouter localement un décalage de tous les éléments affichés pour que le joueur soit au centre de l'écran. C'est donc dans le GameManager que chaque fraction de seconde le manager va récupérer le décalage du joueur local puis, va localement seulement dessiner tous les éléments en rajoutant à leur position ce décalage.
9. Pour centraliser les entités dans le GameState nous avons créé l'EntityManager. Ce module permet de gérer toutes les entités du même type, par exemple, dans le GameState, il y a la liste de tous les joueurs. Cela permet de mettre à jour simplement les entités locales avec celles reçues du serveur. Cela permet aussi de supprimer une entité locale si elle n'est plus présente dans la copie du serveur. De plus, la centralisation des entités permet de toutes les visualiser en un seul bloc.
10. Nous avons ensuite géré les sorts. Grâce aux événements de Pygame nous pouvons lancer un sort avec le bouton de la souris. Les sorts sont gérés comme des entités car ils peuvent mourir quand ils touchent un ennemi ou au bout d'un certain temps. Pour ce faire, quand le joueur appuie sur le bouton de sa souris, cela va envoyer au serveur une demande de création de sort. Après l'avoir créé, le spell va avancer tout droit dans la direction du pointeur de la souris. La mise à jour de sa position se fait du côté du serveur comme le reste des mises à jour. Les sorts ne sont représentés que par des sphères bleues pour la première soutenance. La création des sorts a permis la mise en place de la gestion des événements qui sera utile pour la suite.
11. Par la suite, nous avons rajouté des murs. Cela peut paraître simple, mais ceci a été bien plus compliqué que prévu. Nous avons créé une simple forme géométrique mais nous avons dû lui rajouter une hitbox et gérer les collisions avec celle-ci.

12. Pour gérer les collisions, nous avons créé la classe hitbox et nous l'avons ajoutée aux entités. Les hitbox se servent de Pygame, notamment sa méthode `spritecollide` qui permet de savoir si deux surfaces de Pygame, donc deux éléments, se touchent. Cette information sera alors utilisée pour simuler des collisions.
13. Ensuite, pour créer les collisions entre le joueur et les murs, nous allons à chaque fois que le joueur se déplace regarder s'il se trouve dans la hitbox d'un obstacle et si c'est le cas nous le repositionnerons à sa position d'origine. Ceci a donc permis de délimiter le monde pour tester les collisions et empêcher le joueur de sortir de la carte.
14. Pour simplifier le processus de création de menus et l'interface, toute une méthode de création de menus a été créée. Nous avons centralisé les différents menus dans un seul fichier qui les contient tous. Cependant, pour créer un menu ou une interface, il suffit de suivre un motif dont le contenu du menu doit être une fonction qui renvoie une liste de composants. Chaque composant a été créé manuellement et représente un texte, un bouton, une entrée de texte. Il suffit alors de lister les différents éléments d'une interface pour qu'ils apparaissent automatiquement sans devoir manuellement gérer les surfaces ou autres propriétés de Pygame. Pour ces composants, il y a aussi la possibilité de choisir une position d'ancrage. Cela correspond à quel coin de l'écran le composant doit être ancré. Concrètement, cela permet de placer un texte en haut au milieu de l'écran et la position donnée à cet élément sera relative à ce point. Ceci permet au menu d'être adapté à toutes résolutions. Cet environnement de travail permet aussi de faire des menus dynamiques en permettant de changer un composant pendant que le jeu tourne. En mettant à jour les éléments directement dans la fonction.

Exemple de composant :

```
# adress_input
TextInput(
    "Ip Address",
    position=(0, -100),
    width=200,
    height=50,
    onTextChanged=lambda x: set_val(0, x),
    anchor=Anchor.CENTER,
    initial_text="127.0.0.1",
),
```

15. Pour fluidifier le mouvement des entités, nous avons essayé d'appliquer une interpolation aux entités du côté du client. L'interpolation permet de passer de simplement téléporter l'entité à sa nouvelle position à créer un mouvement local seulement fluide pour masquer la téléportation. Cependant cette partie a été commencée puis mise en suspens car trop complexe et moins importante.
16. Nous avons aussi développé un manager de collisions nous permettant d'effectuer des actions spéciales lors de certaines collisions (ex: la collision entre une attaque et un ennemi inflige des dégâts à l'ennemi).
Ce manager repose principalement sur deux parties, une première étant une liste de dictionnaires réunissant la totalité des collisions possibles avec les deux entités de la collision et la fonction réalisant les opérations de la collision. Elle permet de faire un lien rapide entre une collision entre deux entités et une action ([collisions.py](#)). La deuxième permet de tester s'il y a des collisions et dans ce cas de faire appel à la première partie pour réaliser les collisions (collisionManager.py).
17. Des premières ébauches d'IA ont été réalisées. Une pour les PNJs et une pour les ennemis, celle des ennemis se déplace tout droit vers le joueur le plus proche et celle des PNJs vers une position aléatoire. Cette version était peu lisible et très difficile à comprendre puisqu'elle était intégrée au code des objets des entités.
18. Pour les ennemis et les joueurs nous avons créé un système de vie/dégât. Les dégâts du joueur se transmettent à ses attaques qui sont infligées lors du contact avec un ennemi. Les ennemis ne possédant pas encore d'attaques, leurs dégâts sont pour l'instant inutiles. Cette partie a nécessité l'ajout d'une collision (dans collisions.py).
19. Les ennemis et les PNJs ayant finalement un moyen de déplacement, il a fallu faire en sorte qu'ils arrêtent de traverser les murs, nous leur avons donc ajouté un système de collision avec des murs suivant le même principe énoncé précédemment pour le joueur.
20. Le système d'IA étant complètement brouillon, nous nous sommes penchés sur la création d'un manager d'IA (iaManager.py). Ainsi, nous avons conçu des outils simples, comme la récupération des positions des joueurs, celle du plus proche uniquement ou bien encore le calcul d'une direction grâce à une position.
Avec l'aide de ces outils nous avons pu donner aux PNJ une IA les faisant aller en ligne droite vers une position aléatoire et aux ennemis une IA les faisant se diriger vers le joueur le plus proche. Pour ce faire, nous avons créé une nouvelle classe ia permettant de faire le lien entre une entité et son IA (portant le nom de l'entité).

21. Nous avons donc des déplacements qui étaient effectués mais ceux-ci ne prenaient pas en compte les murs (les entités rentraient dans les murs et glissaient dessus), ce qui nous a amenés à développer et implémenter un système de path finding.

Le système que nous avons choisi d'utiliser est l'algorithme A*. Il repose sur une évaluation des distances à parcourir pour chacune des cases visitées (distance connue depuis la position de départ + distance supposée avec un calcul de la distance à vol d'oiseau ou distance de Manhattan). Nous choisissons ensuite de visiter la case avec la distance la plus faible et en cas d'égalité, celle la plus proche du point d'arrivée et nous ajoutons toutes ses voisines à une liste de cases à visiter. Nous piochons ensuite dans cette liste en respectant les mêmes critères et en suivant les mêmes actions jusqu'à trouver l'arrivée (ou ne plus avoir de cases à visiter).

(voici les vidéos / documentations nous ayant permis de développer et intégrer cet algorithme:

- https://fr.wikipedia.org/wiki/Algorithme_A,
- <https://www.youtube.com/watch?v=iMRJzA8BBIg>

)

22. Pour cette première soutenance, il y a également un prototype de reconnaissance vocale grâce à vosk. Vosk est une bibliothèque Python, qui grâce à un modèle en local permet de transcrire ce que le joueur dit. Cette transcription permet par la suite d'exécuter des actions. Pour la première soutenance, il n'y a que le mot "spell" qui permet de lancer un sort. Cette partie constitue l'identité d'Avada-Kedavoix. Celle-ci fonctionne de la manière suivante : quand le joueur parle, le flux de l'audio est ajouté à une file. Cette file va ensuite être retirée pour extraire les mots retranscrits. Ces mots déchiffrés vont être ajoutés à une nouvelle file globale. Cette file qui contient les paroles du joueur va être triée à chaque mise à jour du GameManager. Le GameManager va alors regarder si les paroles contiennent une des actions enregistrées. Par exemple, si dans les paroles se trouve le mot "spell" le GameManager va déclencher l'apparition d'un sort. Vosk a besoin d'un modèle en local, il en existe de plusieurs tailles et en plusieurs langues mais nous avons pour l'instant utilisé seulement le modèle small en français.

23. Pour ajouter nos graphismes au jeu, nous avons d'abord commencé par mettre en place un module permettant de s'occuper des animations du personnage du joueur. En définissant plusieurs images par animation et en les changeant à intervalles réguliers, cela donne l'illusion que le personnage marche ou reste sur place.

24. Nous avons un pathfinding fonctionnel, toutefois le problème de cette solution est que la recherche d'un chemin nécessite une très grande quantité de calculs, il est donc impossible de rechercher tout le chemin d'un coup dans le jeu, cela créerait des lags trop importants ; alors si la cible se déplace pendant qu'il suit le chemin qui vient d'être créé, il n'arrivera pas au bon endroit.

Nous avons donc pris plusieurs mesures afin d'optimiser l'algorithme. Premièrement, nous avons séparé le calcul sur plusieurs frames (ex : nous calculons cinq positions vers la destination puis, nous nous déplaçons et nous recalculons cinq positions) ce qui résout aussi le problème des déplacements de la cible. La deuxième solution est d'espacer les positions. Nous avons implémenté cette solution avec une position = 1 pixel, nous avons donc créé une carte séparant chaque position de 20 pixels (modifiable). C'est un système facile à implémenter puisqu'il suffit d'arrondir chacune des coordonnées à 20 près.

25. Malgré tout il nous reste deux bugs:

- Premièrement, lors des calculs de positions et de l'accessibilité de la position, il semblerait qu'il ne trouve aucune collision avec des murs pour certaines positions (le problème est en cours de résolution nous avons donc désactivé la physique des ennemis afin de régler ce problème).
- Deuxièmement, comme nous n'effectuons que cinq calculs lors de déplacement les ennemis risquent de rester bloqués derrière certains murs : ils estiment que le chemin le plus rapide pour passer le mur est en allant vers la droite car la cible se trouve sur leur droite puis, passent à gauche de la cible et donc ils changent d'avis ce qui provoque une oscillation entre deux positions.

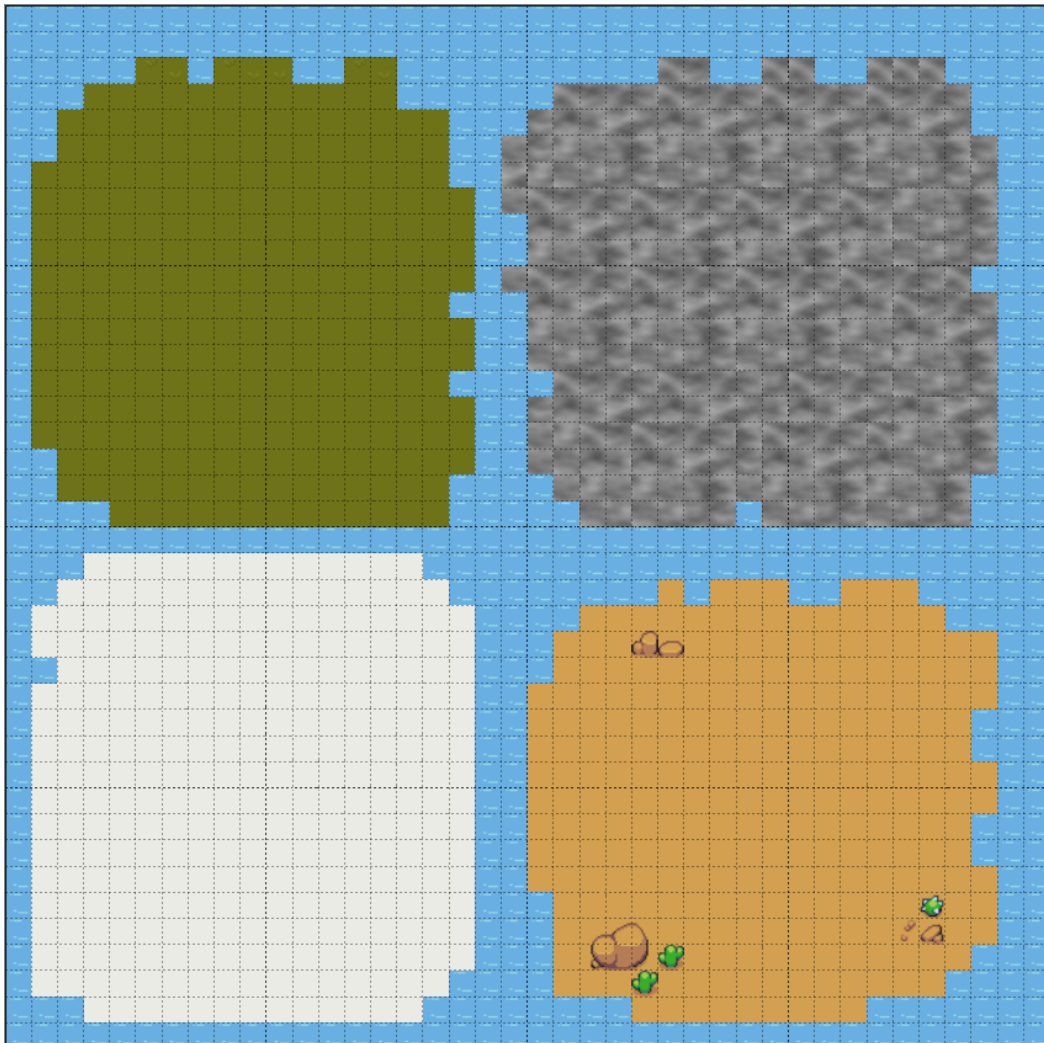
26. Pour la map, nous avons décidé d'utiliser le tilemapping. Pour ce faire, nous nous sommes orientés vers Tiled. Ce logiciel nous permettra d'utiliser des calques afin de superposer les objets et la carte en elle-même et donc de créer un effet de profondeur.

Une fois le logiciel choisi, nous avons commencé à faire des recherches sur des sites comme Craft Pix ou encore [Itch.io](https://itch.io) afin de trouver des ressources graphiques que nous pourrions utiliser.

Nous avons commencé avec une map uniquement constituée de tuiles représentant de l'herbe pour pouvoir faire des tests.

Finalement, la map est toujours en phase de conception mais elle est plus développée. La taille reste encore à être définie. En effet, pour le moment la map a l'air assez petite. De plus, la taille des tuiles doit être pensée car nous avons des ressources en 32X32 pixels ce qui a été donné comme référence au début mais avec les nouvelles recherches établies, nous possédons des

assets en 16X16 pixels. Nous avons donc modifié la map pour qu'elle soit avec des tuiles de 16X16 pixels sauf que les assets de 32X32 pixels ne peuvent plus être utilisés car une tuile de 32X32 pixels se traduit normalement en un bloc de 4 tuiles alors que dans certains cas, ce bloc est représenté uniquement par 2 tuiles collées horizontalement, ce qui nous empêche de pouvoir être plus précis dans ce que nous voulons faire.



27. Pour gérer l'affichage de la map, nous nous sommes servis de la bibliothèque pytmx qui permet de simplement afficher une carte en .tmx, le format des cartes de Tiled. Il a donc suffi de redessiner chaque tuile pour former une grande carte où le joueur peut se déplacer. Pour encapsuler cette logique nous avons encore une fois créé une classe spécifique.

28. Pour le site web, nous avons commencé par regarder la documentation sur le site W3Schools afin de revoir le fonctionnement des balises HTML ainsi que les bases du CSS.

Puis, nous avons conçu la structure de base du site en créant les différentes pages HTML et en les interconnectant entre elles. Nous avons ensuite implémenté un menu de navigation commun à l'ensemble des pages en utilisant les listes HTML.

En parallèle, nous avons utilisé le CSS pour réaliser un visuel simple, que nous souhaiterions améliorer par la suite.

Nous avons commencé à intégrer des informations sur chacune des pages même si celles-ci restent encore à compléter. Nous avons notamment inclus un bouton de téléchargement permettant d'accéder à notre dépôt GitHub sur lequel le jeu est hébergé, ainsi qu'au rapport de la première soutenance au format PDF. Nous avons également commencé la rédaction de la biographie de chaque membre de l'équipe.

Par la suite, nous voudrions ajouter davantage de contenu en rapport avec le jeu tels que la description des ennemis, l'environnement du jeu, l'histoire principale ainsi que d'autres éléments. Nous souhaiterions également rajouter une vidéo de démonstration du jeu à côté du résumé.

Pour réaliser certains visuels du site web, tels que les cartes de présentation des membres de l'équipe avec un effet de survol, nous nous sommes inspirés de différentes vidéos.

Voici quelques liens utilisés :

- [CSS Card Hover Effects | HTML & CSS](#)
- [How To Make Animated Website Design Using HTML And CSS Step By Step](#)

Enfin, pour améliorer le visuel du site, nous avons commencé à réaliser un prototype en utilisant le site Figma. Cela nous permet de visualiser précisément le rendu que nous souhaitons créer avant de passer à l'implémentation du code.

29. Pour les graphismes, nous avons décidé de dessiner nous-mêmes les sorts et les objets de l'inventaire. Concernant les personnages, si le temps nous le permettait, nous souhaiterions également les réaliser nous-mêmes et dans le cas contraire nous utiliserons des ressources trouvées sur Internet.

Nous avons déjà recherché certaines ressources au cas où cette situation se produirait comme celles que vous pourrez trouver en annexe.

En revanche, nous avons encore très peu avancé sur les graphismes à réaliser nous-mêmes car Mathilde H. n'a pu ramener sa tablette graphique que récemment à Toulouse. Cependant, nous avons réalisé un premier prototype de cristal de glace, qui pourra être utilisé dans la création d'un sort de glace.

30. Pour l'histoire, nous avons réalisé tout d'abord des schémas des dialogues, pour être fixés sur quel choix amène à quelle réponse, tout en laissant des options secondaires au cas où ces fonctionnalités ne verraient pas le jour, car nous ne sommes pas encore fixés sur ce point. L'écriture de l'histoire principale avec les explications de fonctionnement du début est presque terminée. Seule la version en français a été réalisée pour le moment, il est prévu qu'il y ait une version en anglais. L'implémentation n'a pas encore été faite, il manque également le cadre dans lequel seront affichées toutes les parties de ce texte. Nous avons un début de graphisme (en annexe) du sol que nous allons utiliser, sous forme de tuiles que l'on va simplement mettre bout à bout jusqu'à obtenir la surface voulue. Les murs seront vraisemblablement faits avec des motifs de pierres similaires.

31. Pour gérer le donjon et donc l'existence de plusieurs niveaux, nous avons utilisé un système de couche. Chaque entité et chaque objet qui est affiché à l'écran possède une variable "world_layer". Localement, chaque joueur ne va afficher que les éléments qui ont la même couche que lui. Par exemple, si le joueur est à la surface, il ne va afficher, faire le rendu que des éléments qui se trouvent aussi à la surface. Ceci a posé plusieurs problèmes, comme par exemple le fait qu'il a fallu adapter le système de collision et de physique mais aussi le système d'ia. Les collisions ont dû être adaptées pour ne vérifier que les collisions entre les éléments d'une même couche. De même, les IA ont dû être adaptées pour rechercher un joueur seulement s'il se trouve sur leur couche.

32. Pour naviguer entre ces couches et donc avancer dans le donjon, nous avons implémenté des portes. Ces portes, lorsque le joueur est assez près, affichent une aide sur l'écran. Cette aide l'invite à appuyer sur la touche d'action pour pouvoir utiliser cette porte. Une fois utilisée, le joueur va simplement avoir sa propriété "world_layer" changée s'il était à la surface et qu'il utilise la porte sa valeur va juste augmenter pour arriver dans le donjon à la même position.

33. Pour générer les différents niveaux du donjon, nous avons créé un script pour générer aléatoirement un motif de salles rectangulaires avec des ouvertures permettant aux joueurs de se balader. Pour ce faire, nous avons d'abord une zone délimitée correspondant à la taille de l'étage et on va essayer de remplir toute la zone avec des rectangles de taille aléatoire représentant les salles. Après avoir créé les salles, il a fallu créer les ouvertures entre celles-ci en prenant en compte que chaque salle devait avoir au minimum une ouverture et que chaque salle devait pouvoir être atteinte. Lorsque le joueur va prendre la première porte, il va rentrer dans le donjon où se trouvera au même endroit une porte permettant de revenir en arrière au même endroit. Pour éviter des problèmes, la zone où la porte apparaît est sécurisée, empêchant un mur d'apparaître sur la porte. Pour pouvoir continuer dans le donjon, chaque niveau va placer dans une salle aléatoire une nouvelle porte menant vers le prochain niveau. On génère donc de manière itérative 10 niveaux.
34. Pour gérer l'apparition des ennemis dans les différents niveaux du donjon nous avons créé la classe `EnemySpawner`. Nous avons manuellement défini une liste d'ennemis par étage et cette classe va donc instancier à des positions aléatoires des ennemis dans le niveau.
35. Pour rendre la navigation dans le donjon plus compliquée, nous avons rajouté un filtre pour donner l'impression d'une faible luminosité. Pour ce faire, on va créer avec pygame un fond noir et placer au centre un cercle transparent avec un fondu entre le transparent et le opaque. Le joueur se voit donc au travers de ce cercle transparent.
36. Pour que les IA puissent fonctionner malgré l'ajout des différentes couches, nous avons simplement rajouté une condition dans la recherche du joueur le plus proche. Chaque ennemi cherche désormais le joueur le plus proche se trouvant dans la même couche que l'entité.
37. Pour gérer les différents sorts, nous avons créé les différents types de sorts et regroupé la logique dans un "`SpellsManager`". Désormais, il existe des sorts qui ne demandent pas de simplement générer un sort visuellement et de le lancer. Nous avons rajouté les sorts utilitaires comme le sort pour se soigner ou encore le sort pour revenir à l'origine du monde à la surface.
38. Certaines entités possèdent aussi une barre de vie. Ces barres de vie qui ne sont rendues que du côté client représentent la proportion de vie restante par rapport à la vie maximale. Cette barre est rendue entièrement via pygame et la taille de la barre rouge représentant la vie va se mettre à jour à chaque fois que l'entité prend des dégâts.

39. La librairie de vosk utilisée pour la reconnaissance vocale posait problème selon les os. Pour palier à ce problème, nous avons donc fait en sorte de télécharger le modèle automatiquement au premier lancement du jeu et le placer localement au bon endroit. Nous avons découvert que le problème venait de l'accès au modèle local donc le fait de le télécharger au bon endroit et de renforcer le chemin d'accès au dossier du modèle a résolu le problème.
40. Résolution bug network: Nous avons oublié de fermer les sockets lors de l'arrêt du jeu ce qui pouvait générer un bug si l'on redémarrait rapidement après la fermeture du jeu vu que le port que l'on essayait de récupérer était encore occupé par le lancement précédent du jeu.
41. Ajout d'un système de debug pour le pathfinding: Nous avons déplacé le stockage du chemin vers l'ennemi permettant d'y accéder plus simplement. Nous avons ensuite créé une fonction permettant l'affichage d'un chemin de points qui va être suivi par l'ennemi (ou l'entité utilisant un pathfinding). Ce qui permet de debug efficacement les problèmes liés au path finding.
42. résolution bug pathfinding: Nous avions précédemment un bug faisant passer les ennemis dans les murs. Il était dû à un mauvais suivi du chemin. Le chemin étant une succession de positions, on effectuait un déplacement vers la première position puis on l'enlevait résultant au suivis de la position suivante sans avoir atteint la première. Nous avons ajouté des tests vérifiant que l'on avait atteint la première position avant de suivre la suivante.
43. Ajout des types d'ennemis: Nous avons ensuite copié le système de types pour les sorts afin de créer les ennemis avec des caractéristiques spécifiques en fonction de leurs types.
Nous avons ajouté 3 types:
-GOBELIN_MASSUE avec une attaque de proximité (un coup de massue).
-DRAGON avec une attaque de distance (une boule de feu).
-SQUELETTE avec une attaque de distance (il a récupéré l'image du dragon et l'attaque de glace mais cela n'est pas définitif).
44. Optimisation pathfinding: Le pathfinding ayant été réparé, nous l'avons optimisé afin de réduire sa consommation de calcul et d'augmenter sa portée, ce qui consiste dans le fait de chercher moins de positions et d'espacer les positions.

Modifications comportement des ias: afin d'éviter que les ennemis nous attaquent depuis l'autre bout de la map, on leur a ajouté un attribut de portée nommé: "reach", leur permettant de suivre uniquement les joueurs à une distance de moins de leurs portée. Et comme certains ont une attaque à distance, on leur a donné un attribut "distance from" faisant que lorsque leur distance à leur cible est inférieure à cette valeur, ils s'arrêtent.

Nous avons ensuite appliqué cela aux différents types d'ennemis:

-GOBELIN_MASSUE : ils ont une portée moyenne et vont jusqu'au contact du joueur

-DRAGON et SQUELETTE : ils ont la même portée mais eux ils s'arrêtent à une certaine distance du joueur

45.Animation des sorts: Nous avons appliqué à tous les sorts des animations. Pour cela, nous avons créé quatre dessins différents et nous les avons fait apparaître les uns après les autres dans le jeu, ce qui crée une animation.

46.Animation des ennemis: Nous avons fait de même pour les ennemis même s'il reste certains graphismes à réaliser.

47.graphismes des ennemis et des PNJ: Nous avons d'abord décidé quels seront les ennemis dans notre jeu : il y aura deux types de gobelins, un avec une massue et l'autre avec un poignard. Il y aura également un dragon qui sera le boss final du donjon, des squelettes et un mage noir. A ce jour, nous avons déjà dessiné les deux types de gobelins, ainsi que le dragon.

Au niveau des personnages non joueurs, nous n'avons encore aucun dessin qui est achevé mais nous avons commencé à dessiner un brouillon. Nous aimerions dessiner au moins un marchand, un fermier et un boulanger. En revanche, les habitations du village sont terminées même s'il va peut-être falloir modifier leur taille lorsqu'on les intégrera dans le jeu.

48.graphismes des sorts: Au niveau des sorts, nous avons dessiné un sort de feu et un sort de glace et nous aimerions en rajouter d'autres par la suite, comme par exemple un sort d'eau ou de foudre. Il faut aussi que nous dessinions l'attaque du dragon qui sera probablement une grosse flamme ou alors nous garderons la même attaque de feu que celle du magicien mais nous augmenteront sa taille.

49.Site web: Nous n'avons pas modifié et amélioré beaucoup de choses sur le site web. Nous avons notamment ajouté une image du jeu, que nous aimerions plus tard remplacer par une vidéo de démonstration. Il nous reste à ajouter la description de chaque membre du groupe, ainsi que les liens vers toutes les ressources que nous avons utilisées pour coder le jeu.

50. Événement de mort: ajout du fait que lorsque notre vie passe en dessous de 0 on se téléporte au tout début (dans la zone du village) du jeu avec la vie pleine et on est obligé de recommencer.
51. Pour ajouter les dialogues des PNJ nous avons créé un menu de dialogue qui apparaît quand on appuie sur la touche d'action près d'un PNJ quand l'événement est détecté cela va afficher le menu de dialogue avec le nom de la personne qui parle et son texte. Un appuie sur le bouton de la souris permet de passer au dialogue suivant. Chaque PNJ peut avoir ses propres dialogues.
52. Il est possible d'avoir des discussions avec les personnages non joueurs du village. Afin d'obtenir des informations, de nouveaux sorts ou de nouvelles compétences. Il y a plusieurs alternatives de conversation en fonction. De la personne en fonction des joueurs et de l'avancée du jeu. Certains des personnages non joueurs donnent accès à de nouveaux sorts. Par le biais d'énigmes, en récupérant un certain nombre d'indices pour déchiffrer un message codé par exemple. Le nouveau mot représente le sort à prononcer. Il est aussi possible de faire un jeu de hasard, ou encore d'acheter un sort avec des pièces, si on en accumule un certain nombre que l'on trouve au cours de la quête dont le labyrinthe. Le joueur pourra potentiellement acheter des compétences qui pourraient être, par exemple, des sorts de soin, d'augmentation de la barre de vie, de rétrécissement du sorcier, de téléportation, pour passer à travers les murs ou d'invisibilité. Ce sont des propositions sur lesquelles nous travaillons en ce moment. En ce qui concerne les messages codés, ce sera principalement des mots ou des chiffres qui ne veulent rien dire et chaque indice sera constitué de la lettre de départ vers sa transcription pour au final, tous les indices combinés avec le mot-clé donneront le mot que l'on pourra utiliser.
53. La première idée était de récolter des objets ou des pièces pour débloquent des sorts après être allé parler aux PNJ or ce système s'est avéré pénible à réaliser et peu intéressant en terme d'expérience de jeu, nous avons essayé de le mettre en place mais tout le monde préférerait finir les niveaux et tuer les ennemis plutôt que s'embêter chercher si on avait pas oublier de récolter des pièces. S'il en manquait une seule le sort n'était pas accessible on pouvait oublier l'emplacement de la porte de sortie en cherchant les pièces ce qui était quelque peu frustrant. Nous avons donc décidé de rester sur de simples petites énigmes pour débloquent des sorts puisque cette fonctionnalité est secondaire par rapport aux combats avec les monstres.

54. Pour permettre au joueur de reprendre une partie en solo ou en hébergement, nous avons ajouté un système de sauvegarde du monde. Ainsi, lorsqu'un joueur choisit de jouer seul ou d'héberger une partie, il peut soit continuer une partie déjà commencée, soit lancer une nouvelle partie à partir de zéro. Cette fonctionnalité a pu être mise en place assez rapidement, car le serveur conserve déjà en mémoire l'état complet du jeu pendant la session, notamment la position du joueur, les ennemis et les autres entités du monde. La sauvegarde consiste donc simplement à sérialiser cet état global en JSON. Lors du chargement, le fichier JSON est relu puis réinjecté dans le GameState du serveur, le client reçoit ensuite cet état et reconstruit localement les entités manquantes ou met à jour celles qui existent déjà, comme lors d'une partie sans sauvegarde. Le principal ajustement a concerné les collisions, car certains obstacles, comme les murs du donjon, ne sont pas restaurés automatiquement à partir du GameState seulement. Il a donc fallu les rajouter manuellement dans les groupes de collision afin de retrouver un comportement cohérent après le chargement.
55. Nous avons également mis en place la génération d'un exécutable au format .exe à l'aide de PyInstaller. Cet exécutable embarque le script Python principal qui lance client/main.py, ainsi que l'ensemble des ressources nécessaires au client, notamment les images, les sons et le modèle de base utilisé pour la reconnaissance vocale.
56. Pour améliorer l'expérience utilisateur, nous avons repensé le système de plein écran en utilisant le flag NOFRAME, qui permet d'occuper l'intégralité de l'espace d'affichage disponible. Par ailleurs, nous avons ajouté la touche F11, offrant à l'utilisateur la possibilité d'activer ou de désactiver le mode plein écran à tout moment.
57. Nous avons gardé la même logique que pour le menu d'accueil. Cependant ici, on retrouve les différentes touches utiles pour le joueur ainsi que des boutons pour sortir du menu et quitter le jeu. Les touches ne sont malheureusement pas changeables. Nous ne pouvons pas non plus changer l'entrée du micro directement dans ces paramètres là. Il va falloir le faire directement sur votre PC. Puisqu'il y a eu certains problèmes liés avec SOUNDDEVICE.
58. Nous avons mis en place un cycle jour/nuit afin de renforcer l'immersion dans le jeu. Ce cycle alterne une phase de cinq minutes de jour et une phase de cinq minutes de nuit. Pour simuler l'obscurité, une surface noire semi-transparente est appliquée sur l'ensemble de l'écran à l'aide d'un mode de fusion, ce qui crée l'illusion de la nuit. De plus, au début de la nuit, certains ennemis peuvent apparaître aléatoirement à la surface lorsqu'au moins un joueur s'y trouve.

59. Nous avons ajouté des éléments d'interface à l'écran lorsque le joueur explore le donjon afin de lui fournir davantage d'informations. Dans un premier temps, nous affichons en haut et au centre les données liées au niveau actuel, comme l'étage en cours et le nombre d'ennemis encore en vie. De plus, lorsque le joueur entre dans la salle du boss située au dernier niveau, ces informations sont remplacées par la barre de vie du boss. Le boss ne possède donc pas simplement une barre de vie au-dessus de sa tête, mais un élément dédié dans l'interface.

60. Un système de gestion sonore a été développé à l'aide de Pygame afin de centraliser la lecture des effets sonores et de la musique. Toute la logique est regroupée dans le gestionnaire sonore, ce qui permet de lancer une musique de fond ou de jouer ponctuellement un effet sonore, par exemple lors d'un clic sur un bouton dans l'interface. Les effets sonores et la musique sont traités séparément, ce qui rend possible un réglage indépendant de leurs volumes. Pour optimiser l'utilisation du système, les fichiers audio au format .mp3 sont chargés au démarrage puis stockés dans des dictionnaires associés à leur nom, ce qui permet de déclencher facilement le son correspondant au moment voulu.
61. Afin d'optimiser les échanges réseau côté serveur, nous avons mis en place un envoi différencié des données en fonction de la couche de jeu actuellement occupée par chaque joueur. Ainsi, le serveur ne transmet pas l'intégralité du monde à tous les clients, mais uniquement l'état correspondant à la couche concernée, ce qui réduit la quantité de données envoyées, diminue la charge réseau et améliore la fluidité générale, en particulier lorsque plusieurs joueurs sont connectés simultanément. Cette approche implique toutefois une gestion plus fine de la synchronisation, car le serveur doit maintenir une représentation cohérente de chaque couche et actualiser dynamiquement les entités visibles lors des changements de niveau. Côté client, cela suppose également de reconstruire ou de mettre à jour localement uniquement les éléments utiles à la couche reçue, ce qui rend les transitions plus légères mais aussi plus dépendantes de la bonne synchronisation entre serveur et client.
62. L'écran de chargement a été ajouté afin de limiter au maximum le décalage entre le moment où le joueur peut commencer à se déplacer et celui où l'ensemble des entités du monde est correctement synchronisé. Ce problème vient du fait qu'au lancement d'une partie, le client doit à la fois se connecter au serveur, recevoir l'état initial du monde et charger les ressources nécessaires à l'affichage, ce qui peut entraîner des problèmes de synchronisation avec le serveur et laisser les entités immobiles pendant un certain temps. Pour éviter cela, le GameManager active un état de chargement temporaire côté client. Cet état est maintenu jusqu'à la réception du premier état complet envoyé par le serveur. Pendant cette phase, l'interface de chargement est affichée, les images sont préchargées et le rendu de la partie est suspendu, ce qui permet de masquer la synchronisation initiale et de faire apparaître le jeu uniquement lorsque le client dispose déjà d'un monde exploitable. La même méthode est utilisée lors des changements de couche, car le serveur n'envoie alors que les informations relatives à la couche du joueur, ce qui peut provoquer un court temps de chargement lors du basculement vers la nouvelle couche. Pour résoudre ce problème, l'écran

de chargement est affiché et le serveur rend le joueur invincible pendant un court instant, évitant ainsi qu'il ne se fasse tuer durant le chargement.

63. L'écran de crédit permet de présenter les personnes ayant contribué au projet ainsi que les remerciements et ressources utilisées. Il s'agit d'un menu simple accessible sur le menu principal et se présentant sous la forme d'un générique de fin de film.
64. Afin de corriger le problème empêchant le lancement du serveur seul via `python server/main.py`, nous avons adapté la structure du projet pour qu'elle soit compatible avec une exécution directe depuis le dossier serveur. Les chemins d'import ont été corrigés afin qu'ils ne dépendent plus du répertoire courant, ce qui supprimait les erreurs liées à la résolution des modules. En parallèle, certaines classes partagées entre le client et le serveur ont été ajustées pour distinguer clairement les comportements côté serveur et côté client grâce à un mode dédié, ce qui évite au serveur d'initialiser des éléments graphiques inutiles.
65. La synchronisation des animations et de l'orientation des entités a été corrigée, à chaque modification locale de position, la direction du sprite est recalculée et le sprite est inversé selon que l'entité se déplace vers la gauche ou vers la droite, garantissant ainsi une correspondance visuelle immédiate entre mouvement et animation.
66. La réapparition du joueur est gérée de manière centralisée afin de garantir une remise en état propre entre le serveur et le client. Quand le personnage meurt, le serveur prépare une réinitialisation complète de son état, puis renvoie au client les données nécessaires pour recréer un état cohérent du monde. De son côté, le client applique cette mise à jour, replace le joueur à son point de départ et réactive l'affichage normal. Ce fonctionnement évite les incohérences après la mort, car la reprise se fait toujours à partir de l'état validé par le serveur.
67. Des nouveaux sorts ont été développés comme le sort de soin qui permet lorsque l'on récite l'incantation de soigner. De plus, il y a le sort de téléportation qui permet au joueur de se téléporter au centre du monde au point d'apparition de base du personnage. Enfin, nous avons modifié le sort de la boule de feu pour en faire un sort de boule de feu sombre lancé par certains ennemis.

68. Comme nous avons des menus rudimentaires ou non fini nous en avons ajouté des boutons:

- un bouton Exit ou BACK dans chaque menu permettant comme le noms l'indique de fermer le jeu ou de retourner sur le menu principal
- un bouton menant vers les crédits (déjà mentionné précédemment)
- un bouton menant vers un nouveau menu SETTINGS

Nous avons quelques bugs dans les menus comme un fond d'écran qui ne s'ajustait pas à la taille de la fenêtre ou bien des boutons qui sortaient de l'écran, nous avons donc dû les résoudre.

69. Le menu "SETTINGS" qui vient d'être énoncé est un menu permettant de visionner les touches permettant le déplacement.

70. La quête pour avoir le sort de glace est une devinette où il faut tester différents sorts donnés en discutant avec le marchand, sinon il faut vaincre le plus d'ennemis possible pour gagner de nouveaux sorts.

71. Nous avons effectué un équilibrage au niveau de la vitesse des ennemis de leurs vie et de leurs dégâts afin de permettre de rendre le jeu plus difficile sur certains aspect plus facile sur d'autres, et surtout de rendre quasi obligatoire le fait de jouer en équipe

72. Nous avons terminé de dessiner tous les graphismes que ce soient les sorts, les personnages non joueurs et les ennemis.

Finalement, nous avons dessiné cinq types d'ennemi : deux types de gobelins, un mage noir, un dragon et un squelette. Ils utilisent chacun des sorts différents pour attaquer le joueur. Celui-ci a d'ailleurs accès à cinq types de sorts différents : une boule de feu, de la glace, un sort de soin, un sort de téléportation et un coup de poing.

De plus, il peut communiquer avec trois personnages non joueurs différents: une boulangère, un marchand et un paysan.

73. Nous avons apporté quelques modifications au site web. Nous avons notamment essayé d'améliorer le visuel de la page de présentation de notre groupe car les colonnes n'étaient pas centrées. En effet, il n'y a que deux images sur la première ligne alors qu'il y en a trois sur la deuxième mais cela créait un espace vide sur la page. Pour remédier à cela, nous avons décidé de créer deux tableaux avec la largeur des colonnes de même taille, un avec deux colonnes et un autre avec trois colonnes. Cela nous a semblé être une solution plus simple que l'utilisation de grid en css.

De plus, nous avons créé les pages avec la description plus précise de chaque membre du groupe, ainsi que la page avec les sources utilisées pour la conception de notre projet tout au long de l'année.

74. Le boss a subit des améliorations, désormais il a pris le skin du mage noir et possède cinq sorts: les sorts de base (feu / glace / feu noir), un sort de téléportation qui lui permet de bouger dans des positions préenregistrées et un sort de soin avec un délai très long lui permettant de se soigner de temps en temps.

Pour ce faire, nous n'avons pas eu à modifier l'ia mais uniquement à créer une logique du lancement des sorts spécialement pour lui. Le sort de soin est détaché des autres et a un compteur distinct mais les quatre autres sorts s'alternent en boucle. Ils suivent ce schéma:

début -> feu -> glace -> téléportation -> feu noir -> début

75. Avec la création des maisons mentionnées plus haut, nous avons décidé de leur ajouter une fonctionnalité, celle de lier les pnjs à une maison. Pour ce faire, nous avons limité la zone de déplacement des pnjs à une zone de 300 pixels autour de celle-ci.

76. Malgré l'existence de différents sorts déblocables et disponibles pour les ennemis, il n'y avait pas de différence dans les effets des sorts (entre la boule de feu et le sort de glace plus particulièrement). Nous avons donc pris la décision de créer un effet de ralentissement pouvant s'appliquer à la fois sur les ennemis et les joueurs.

Pour ce faire, nous avons rajouté des paramètres aux sorts et aux entités permettant de stocker l'intensité de l'effet ainsi que son temps de durée. Ensuite le transfert du sort vers la cible de l'effet est effectué lors de la collision et est appliqué au moment d'appliquer les déplacements.

Problèmes rencontrés

Problème de séparation logique serveur/client:

Le problème de la séparation logique entre le serveur et le client a été présent dès le début du projet. Nous avons eu du mal à distinguer clairement les responsabilités de chacun, car dans notre approche, le serveur joue le rôle de maître du jeu, il met à jour l'ensemble des entités et possède la copie réelle du monde en temps réel, tandis que le client ne gère que sa position locale. Cela a posé problème à plusieurs reprises, notamment lors de la gestion des collisions. Nous avons donc choisi de laisser le serveur s'en occuper, mais afin d'éviter que le joueur local ne subisse de la latence et ne traverse les murs en local, le client vérifie également les collisions avec les murs. Cette approche, assez confuse, reste à améliorer, mais elle a permis de résoudre une partie de nos problèmes.

Le fait que le client gère sa propre position nous a aussi posé des difficultés. En effet, comme le client contrôle sa position et que le serveur s'occupe des dégâts et des points de vie, nous avons eu du mal à replacer correctement le joueur à son point d'apparition lorsqu'il mourait. Pour cela, le serveur envoie un message au client afin de lui indiquer qu'il doit se repositionner à des coordonnées précises. Nous avons également dû mettre en place des interpolations afin d'éviter de voir les entités se déplacer de manière saccadée. En effet pour éviter de mettre à jour les positions des entités seulement quand le client reçoit la position du serveur le client va localement calculer des positions intermédiaires pour fluidifier les mouvements. Mais cette méthode a été compliquée à mettre en place car concrètement au début cela n'enlève pas les mouvements saccadés et c'est donc après un moment à ajuster les paramètres que nous avons réussi à fluidifier les mouvements.

Problème de surcharge:

Nous avons eu des problèmes avec le réseau après l'ajout du donjon, car avant, tous les niveaux étaient envoyés aux clients. Par conséquent, dans certains cas la connexion entre le client et le serveur pouvait être surchargée et se fermer. Pour résoudre ce problème, nous avons donc changé la méthode de communication avec le client pour n'envoyer que les informations sur le niveau où il se trouve.

Création de l'exécutable:

Pour créer l'exécutable du jeu, nous avons été confrontés à des problèmes de librairies et de chemins d'accès, notamment à cause du passage du projet dans un exécutable PyInstaller. Il a donc fallu adapter la configuration pour inclure correctement les dépendances externes, les modules du projet et certaines ressources du jeu, ainsi que corriger la gestion des imports et des chemins relatifs. Nous avons aussi dû ajuster le script de build afin qu'il crée automatiquement l'environnement virtuel, installe les dépendances nécessaires et lance la génération de l'exécutable dans de bonnes conditions.

Merge conflicts:

Malgré que nous effectuons des merge assez souvent pour éviter les merge conflicts, il nous est arrivé d'être confrontés à ce genre de problèmes où deux personnes ont modifiés des parties proches du code. Par exemple, quand nous avons dû merge le code de l'ajout des graphismes, qui a impliqué la création de nouveaux ennemis et en même temps la modification de l'ia des ennemis, les deux branches ont modifié la liste des ennemis disponibles. Par conséquent, nous avons dû manuellement faire un mix et adapter le code pour pouvoir réaliser ce merge.

Problème de compatibilité :

Comme certains développent le jeu sous windows et d'autre sous linux nous avons été confrontés à des problèmes inattendus de chemin d'accès vers les ressources (graphiques principalement). En effet nous avons donc découvert que les chemins relatifs ne sont pas gérés de la même manière selon le système d'exploitation. Pour ce faire, nous avons modifié la manière dont nous créons les chemins relatifs pour résoudre ce problème.

Problème sur les menus :

Lorsque nous avons créé les menus, nous avons mis une image en fond avec une taille fixe ce qui posait problème lorsque nous avons codé la possibilité de changer la taille des menus car l'image ne rentrait plus dans le cadre ou bien était trop petite. Nous avons donc dû créer un système permettant le changement de taille du fond. Malheureusement ceci a généré un autre problème: lorsque nous changions de taille de fenêtre, seul l'image du menu actuel était modifiée et comme nous avons une image chargée plusieurs fois pour chaque menu, nous avons été obligés de changer toutes les tailles de tous les menus ce qui peut faire bugger le jeu lorsque cette action est effectuée au mauvais moment (lors du chargement de la partie par exemple).

Problème sur les IAs :

Nous avons utilisé l'algorithme de path finding A* dans les IAs des ennemis. Cet Algorithme nous donnait un chemin de 20 positions espacées d'environ 10 cases. Lorsque l'ia suivait ce chemin, nous nous sommes rendus compte qu'elle passait à travers les murs. La première supposition était que les positions du chemin étaient trop écartées mais même lorsque nous les rapprochions, le problème restait le même. Après mûres réflexions, nous nous sommes rendus compte que le problème venait du fait que même sans avoir atteint la première case, l'IA commençait à suivre la deuxième case. Nous avons donc créé un système permettant de vérifier que la première case était atteinte avant de suivre la suivante.

Problème technique :

Un des ordinateurs que l'on utilisait à été rendu hors d'usage et le seul autre disponible étant ancien, sans possibilité de le remplacer et malheureusement pas compatible avec les systèmes que nous utilisons et le support git qui générait beaucoup trop d'erreurs et l'impossibilité de push. Nous avons été obligés parfois de coder directement sur github ou de partager le code pour qu'il soit push par d'autres personnes du projet, souvent Théo.

État lors de la dernière soutenance

Lors de la dernière soutenance nos principaux objectifs ont été accomplis. Le jeu est jouable seul ou en multijoueur, le joueur peut parler aux villageois et avancer dans le donjon d'étages en étages pour arriver au dernier niveau où se trouve le boss final. Le jeu peut être lancé via un exécutable (.exe) pour arriver sur un menu principal rempli permettant de lancer une partie seul ou a plusieurs a partir d'une nouvelle sauvegarde ou continuer sa précédente partie. Nous pouvons aussi depuis ce menu principal accéder aux paramètres ou rejoindre une partie hébergée par un autre joueur ou hébergée sur un serveur.

Le jeu offre un terrain de jeu où le joueur peut discuter aux villageois, explorer le donjon et lancer les sorts en les sélectionnant et en récitant à voix haute l'incantation appropriée. Dans le donjon se trouve une variété d'ennemis empêchant le joueur de s'engouffrer encore plus profondément dans le donjon.

Au dernier niveau du donjon se trouve un Boss ayant plus de vie que les autres ennemis. Il possède une barre de vie qui s'affiche directement sur l'interface du joueur.

Projection future

A l'avenir nous aimerions développer plus de sorts, pas seulement offensifs mais aussi utiles pour permettre au joueur de réaliser des combinaisons de sorts différents pour avancer plus efficacement ou même lancer des sorts combinés avec les autres joueurs pour renforcer la coopération.

L'élargissement de la carte est envisageable avec différents types de terrain, chacun ayant sa propre histoire et permettant de récupérer ces fameux nouveaux sorts. Nous aimerions également élargir le village et rajouter des villageois avec qui le joueur pourrait communiquer.

Nous pourrions aussi augmenter le nombre de types d'ennemis, ainsi que le nombre de niveaux afin d'avoir un gameplay plus complexe.

Nouvelles connaissances acquises

Mathilde H. :

Grâce à ce projet, j'ai pu revoir les bases du langage HTML et du CSS. J'ai d'ailleurs pu apprendre de nouvelles choses, notamment à réaliser des effets visuels en utilisant le CSS.

De plus, j'ai appris à utiliser GitHub, un outil collaboratif très utile, avec les différentes commandes. J'ai pu comprendre l'utilité des branches dans un projet de groupe.

J'ai également appris à dessiner avec une tablette graphique et à intégrer ces graphismes dans le jeu.

Finalement, ce projet de groupe m'a permis de renforcer ma capacité à travailler en équipe, surtout sur une période aussi longue.

Théo P. :

Via la gestion de la partie réseau, j'ai pu apprendre à gérer la connexion simultanée de plusieurs clients sur un même serveur. J'ai donc appris à sérialiser les données, à encapsuler l'information et à la traiter des deux côtés. J'ai également appris à construire une architecture basée sur la communication entre le serveur et les clients.

De plus, j'ai pu approfondir mes connaissances de Git et GitHub dans un contexte de projet collaboratif, ainsi que l'importance de rédiger un code clair et commenté. Ainsi, j'ai dû gérer plusieurs branches et être confronté à des conflits de merge.

J'ai également appris à utiliser un modèle local d'intelligence artificielle de reconnaissance vocale avec Vosk.

Anthon N. :

Avec ce projet, nous avons pu apprendre de nouvelles compétences. Autant techniques comme l'utilisation de Git grâce aux cours de prog ou encore en discutant entre nous pour les merges et autres. Mais également l'organisation des fichiers pour ne pas se perdre, l'utilisation de POO pour mener à bien le projet, qui n'est pas très intuitif au tout début du projet.

Et ensuite des compétences personnelles afin que nous puissions arriver au bout du projet sans forcément se battre.

Mathilde P. :

Grâce à ce projet j'ai pu développer des compétences techniques sur python, l'utilisation des objets et des classes, ainsi que sur l'utilisation de Git. Le fait que ce projet soit en groupe m'a aidé à fonctionner en équipe, établir des réunions et

respecter les délais du mieux possible. J'ai dû apprendre à me débrouiller par moi-même suite à de très nombreux problèmes techniques et trouver des solutions pour continuer à travailler, ne pas prendre de retard sur le projet et ne pas retarder l'avancement du groupe.

Clément B :

Sur le plan technique j'ai pu développer mes connaissances de l'utilisation des objets sur python ainsi que découvrir l'algorithme A*. J'ai aussi travaillé sur les différentes méthodes d'optimisation (comme ne pas charger ou bouger ce qui n'appartient pas à l'écran ou séparer les calculs en différentes frames). J'ai également appris à utiliser git plus en profondeur.

Sur le plan "travail d'équipe" mon rôle de chef d'équipe m'a permis de comprendre les difficultés de ce rôle (par exemple: motiver les personnes moins impliquées). J'ai aussi pu comprendre l'importance de séparer les tâches entre les personnes et de mettre des objectifs faciles et délimités dans le temps (au niveau personnel et de l'équipe).

Conclusion

Pour conclure, ce projet nous a permis de renforcer de nombreuses compétences, tant sur le plan technique que sur le plan collaboratif.

Nous avons en effet compris les difficultés qu'il peut y avoir en travaillant au sein d'un groupe, notamment sur un projet où nous devons être assez autonomes. Nous avons également compris qu'il était important d'organiser des réunions régulièrement pour faire le point sur l'avancement du projet et discuter des difficultés rencontrées par chacun.

Nous avons réussi à remplir nos principaux objectifs techniques pour le jeu même si des améliorations pourraient encore être apportées.

Les Annexes

- Architecture du projet

```
1  avada-kedavoix/
2  └─ client/                                # Code côté client
3     └─ main.py                            # Point d'entrée du client
4     └─ clientManager.py                   # Connexion réseau et messages
5     └─ gameManager.py                     # Gestion globale du jeu (singleton)
6     └─ menus.py                           # Menus (main, join, host)
7     └─ layerList.py                       # Gestion des couches de rendu
8     └─ runtime.py                         # Boucle/runtime client
9     └─ spellsManager.py                   # Gestion des sorts côté client
10    └─ classes/
11       └─ player.py                       # Joueur (animations, mouvements)
12       └─ enemy.py                       # Ennemi
13       └─ pnj.py                         # PNJ
14       └─ spell.py                       # Sorts et hitbox
15       └─ wall.py                        # Obstacles et murs
16       └─ mapBackground.py               # Fond de carte
17       └─ animator.py                    # Système d'animations
18       └─ hitbox.py                      # Gestion des collisions
19       └─ CameraBlackFade.py             # Effet de transition visuelle
20       └─ clientOnly/
21          └─ clientElements.py           # Éléments spécifiques client
22          └─ dungeonEntrance.py          # Entrée de donjon
23          └─ healthBar.py                # Barre de vie
24    └─ enums/
25       └─ anchor.py                      # Positionnement UI
26    └─ ressources/
27       └─ Dungeon/
28       └─ Enemy/
29       └─ Sorts/
30       └─ UI/
31       └─ Village/
32       └─ wizard-test/
33       └─ tiles/
34    └─ Utils/
35       └─ ImageTool.py                   # Outils de traitement d'image
36    └─ voice/
37       └─ realtimeVoice.py               # Reconnaissance vocale en temps réel
38       └─ vosk-model-small-fr-0.22/
39
40    └─ server/                            # Code côté serveur
41       └─ main.py                        # Entrée serveur et boucle réseau
42       └─ NetworkManager.py              # Gestion des connexions socket
43       └─ gameState.py                   # État global du jeu
44       └─ message.py                     # Protocole et types de messages
45       └─ collisions.py                  # Détection des collisions
46       └─ classes/
47          └─ entity.py                   # Entité serveur
48          └─ serializable.py             # Base sérialisable
49       └─ ia/
50          └─ pathFinding.py              # Pathfinding
51       └─ managers/
52          └─ entityManager.py             # Gestion des entités
53          └─ collisionManager.py          # Gestion des collisions
54          └─ iaManager.py                # Gestion IA ennemis
55          └─ enemySpawner.py             # Spawner d'ennemis
56       └─ world_elements/
57          └─ dungeonWalls.py             # Éléments de monde (murs)
58
59    └─ ui/                                # Système UI partagé
60       └─ UI.py
61       └─ button.py
62       └─ dialogBox.py
63       └─ hotbar.py
64       └─ image.py
65       └─ text.py
66       └─ textInput.py
67       └─ uiUtils.py
68
69    └─ website/                            # Pages web du projet
70       └─ page_du_rapport.html
71       └─ page_des_developpeurs.html
72       └─ style.css
73       └─ images/
74       └─ pdf/
75       └─ video/
76
77    └─ index.html                          # Page web racine
78    └─ README.md                          # Documentation du projet
79    └─ CONTRIBUTING.md                    # Guide de contribution
80    └─ CODE_IMPROVEMENTS.md              # Suggestions d'améliorations
81    └─ .gitignore                         # Fichiers à ignorer
```

- Ressources graphiques



Ressource graphique trouvée sur Internet représentant un magicien de classe glace.



Ressource graphique trouvée sur Internet représentant un magicien de classe feu



sort de feu



attaque de glace



gobelin avec le poignard



gobelin avec la massue



maison



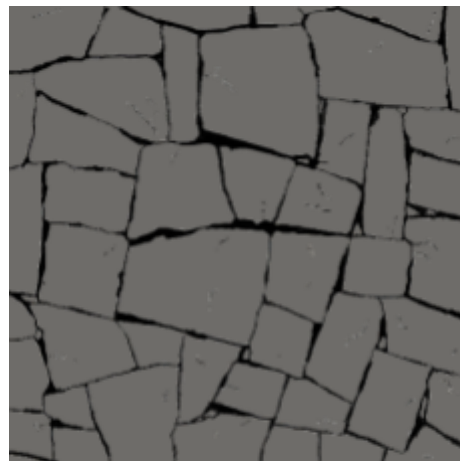
dragon



pièce à utiliser
avec les PNJ



prototype de sorcier



exemple de sol (non retenu)



mage noir



Boulangère



Squelette

- Le planning

Octobre	Novembre	Décembre
Lu Ma Me Je Ve Sa Di	Lu Ma Me Je Ve Sa Di	Lu Ma Me Je Ve Sa Di
1 2 3 4	1	1 2 3 4 5 6
5 6 7 8 9 10 11	2 3 4 5 6 7 8	7 8 9 10 11 12 13
12 13 14 15 16 17 18	9 10 11 12 13 14 15	14 15 16 17 18 19 20
19 20 21 22 23 24 25	16 17 18 19 20 21 22	21 22 23 24 25 26 27
26 27 28 29 30 31	23 24 25 26 27 28 29	28 29 30 31
	30	

Janvier	Février	Mars
Lu Ma Me Je Ve Sa Di	Lu Ma Me Je Ve Sa Di	Lu Ma Me Je Ve Sa Di
1 2 3 4	1	1
5 6 7 8 9 10 11	2 3 4 5 6 7 8	2 3 4 5 6 7 8
12 13 14 15 16 17 18	9 10 11 12 13 14 15	9 10 11 12 13 14 15
19 20 21 22 23 24 25	16 17 18 19 20 21 22	16 17 18 19 20 21 22
26 27 28 29 30 31	23 24 25 26 27 28	23 24 25 26 27 28 29
		30 31

Avril	Mai	Juin
Lu Ma Me Je Ve Sa Di	Lu Ma Me Je Ve Sa Di	Lu Ma Me Je Ve Sa Di
1 2 3 4 5	1 2 3	1 2 3 4 5 6 7
6 7 8 9 10 11 12	4 5 6 7 8 9 10	8 9 10 11 12 13 14
13 14 15 16 17 18 19	11 12 13 14 15 16 17	15 16 17 18 19 20 21
20 21 22 23 24 25 26	18 19 20 21 22 23 24	22 23 24 25 26 27 28
27 28 29 30	25 26 27 28 29 30 31	29 30

Rouge: réunions

Bleu : soutenances orales

Vert : entraînements pour les oraux

- Site web



- Tile assets

